



Universitatea Tehnică „Gheorghe Asachi Iași”
Facultatea de Automatică și Calculatoare
Specializarea Calculatoare și Tehnologia Informației
Disciplina Ingineria Programării



Aplicație quiz auto „IaPermis”

Îndrumător:

Ș.l.dr.ing. Cîmpanu Corina

Autori:

Șalaru Ioana | 1309B

Panaite Elena – Alexandra | 1309B

Vrînceanu Maria – Iustina | 1308A

David Bianca – Vasilica | 1308A

Cuprins

Cuprins.....	2
I. Specificații de Cerințe Software (SRS).....	3
A. Introducere	3
1. Scopul	3
2. Convențiile documentului	3
3. Publicul țintă și sugestii de citire.....	3
4. Domeniul de aplicare al produsului.....	4
5. Referințe.....	4
B. Descriere generală a proiectului.....	4
1. Perspectiva aplicației.....	4
2. Funcțiile produsului	5
3. Clase de utilizatori și caracteristici.....	5
4. Mediu de operare	5
5. Restricții privind proiectarea și implementarea.....	6
6. Documentația utilizatorului.....	6
7. Ipoteze și dependențe	6
C. Cerințe de interfață externă.....	7
1. Interfața cu utilizatorul.....	7
2. Interfața hardware	7
3. Interfața software	8
4. Interfața de comunicații	8
D. Funcțiile sistemului.....	8
1. Autentificare și înregistrare	8
1.1 Descriere și prioritate	8
1.2 Secvențe stimul-răspuns	8
1.3 Cerințe funcționale	8
2. Generarea unui quiz	9
2.1 Descriere și prioritate	9
2.2 Secvențe stimul-răspuns	9
2.3 Cerințe funcționale	9
3. Rezolvarea chestionarului.....	9
3.1 Descriere și prioritate	9
3.2 Secvențe stimul-răspuns	9
3.3 Cerințe funcționale	9
4. Vizualizarea statisticilor	10
4.1 Descriere și prioritate	10
4.2 Secvențe stimul-răspuns	10
4.3 Cerințe funcționale	10
5. Vizualizare rezultate quiz.....	10
5.1 Descriere și prioritate	10
5.2 Secvențe stimul-răspuns	10
5.3 Cerințe funcționale	10
E. Alte cerințe non-funcționale	11
1. Cerințe de performanță.....	11
2. Cerințe de siguranță	11
3. Cerințe de securitate.....	11
5. Reguli de business.....	11
II. Diagrame UML.....	12
A. Diagrama de clase.....	12
B. Diagrama de activități	12

C. Diagrama de secvențe.....	14
D. Diagrama cazuri de utilizare	15
III. Modul de utilizare a programului.....	15
1. Autentificare și creare cont	15
2. Meniu principal.....	16
3. Chestionar.....	16
4. Vizualizarea statisticilor	18
5. Vizualizare rezultat quiz	18
IV. Anexă – Cod sursă relevant & teste	20
V. Listă task-uri îndeplinite.....	31
1. Șalaru Ioana: Interfața Grafică (GUI) și Sistemul de Asistență (Help)	31
2. David Bianca - Vasilica: Arhitectură, Modularizare (DLL) și Șabloane de Proiectare....	31
3. Vrînceanu Iustina Maria: Testare Unitară, Documentarea Codului și Standarde de Scriere	32
4. Panaite Elena Alexandra: Documentația Tehnică și Diagramele UML	32

I. Specificații de Cerințe Software (SRS)

A. Introducere

1. Scopul

Acest document descrie cerințele software pentru aplicația „IaPermis”, o aplicație desktop destinată simulării examenului teoretic pentru obținerea permisului de conducere categoria B. Scopul principal al aplicației este de a oferi utilizatorilor o metodă eficientă și intuitivă de exersare a chestionarelor auto, într-un mod asemănător examenului oficial.

Aplicația permite autentificarea și înregistrarea utilizatorilor, generarea automată a unui set de 26 de întrebări, cronometrarea fiecărei întrebări și afișarea rezultatelor finale.

Aplicația este destinată persoanelor care doresc să se pregătească pentru examenul auto categoria B prin intermediul unei interfețe simple și accesibile.

2. Convențiile documentului

Documentul respectă recomandările IEEE pentru redactarea specificațiilor software. Structura este organizată pe capitole și subcapitole pentru a facilita înțelegerea cerințelor funcționale și non-funcționale. Terminologia utilizată este specifică dezvoltării software și domeniului aplicațiilor educaționale. Cerințele sunt formulate clar și concis, folosind timpul prezent pentru descrierea comportamentului sistemului.

3. Publicul țintă și sugestii de citire

Aplicația IaPermis se adresează persoanelor care se pregătesc pentru obținerea permisului de conducere categoria B.

Documentația este utilă pentru:

- dezvoltatori software;
- testeri;
- evaluatori academici;
- utilizatori interesați de funcționalitățile aplicației.

Pentru o înțelegere generală se recomandă citirea capitolelor introductive și a descrierii generale. Pentru implementare și mentenanță sunt utile capitolele tehnice și diagramele UML.

4. Domeniul de aplicare al produsului

Aplicația se încadrează în domeniul software educațional și al simulatoarelor pentru examene auto.

Scopul aplicației este de a ajuta utilizatorii să se pregătească pentru examenul teoretic auto prin simularea unui chestionar real.

Beneficiile aplicației includ:

- exersarea rapidă a întrebărilor oficiale;
- evaluarea nivelului de pregătire;
- monitorizarea progresului utilizatorului;
- familiarizarea cu structura examenului auto.

Aplicația oferă o experiență apropiată de examenul real prin:

- limitarea timpului pentru fiecare întrebare;
- generarea aleatorie a întrebărilor;
- calcularea automată a rezultatului final.

5. Referințe

- Laborator 3 - Ingineria Programării (reutilizarea codului cu ajutorul DLL-urilor)
- Laborator 4 - Ingineria Programării (fișiere help);
- Laborator 5 - Ingineria Programării (diagrame UML);
- Laborator 12 - Ingineria Programării (testarea unităților);
- The Institute of Electrical and Electronics Engineers – **IEEE Recommended Practice for Software Requirements Specifications** – IEEE Std 830-1993, Inc. 345 East 47th Street, New York, 10017-2394, USA;

B. Descriere generală a proiectului

1. Perspectiva aplicației

IaPermis este o aplicație desktop independentă dezvoltată în limbajul C# utilizând .NET 8 și Windows Forms. Aplicația rulează local pe sistemele de operare Windows și nu necesită conexiune permanentă la internet.

Datele aplicației sunt stocate în fișiere locale:

- fișier XML pentru întrebări;
- fișiere text pentru statistici și utilizatori.

Arhitectura aplicației este modulară și separă logica aplicației de interfața grafică și de mecanismele de stocare a datelor.

2. Funcțiile produsului

Funcționalitățile principale ale aplicației sunt:

- autentificarea utilizatorului;
- creare cont nou;
- generarea aleatorie a unui quiz;
- afișarea întrebărilor și a variantelor de răspuns;
- cronometrarea fiecărei întrebări;
- afișarea imaginilor asociate întrebărilor;
- calcularea scorului final;
- afișarea numărului de greșeli;
- salvarea rezultatelor utilizatorului;
- afișarea statisticilor;
- secțiune Help;
- delogare și ieșire din aplicație

3. Clase de utilizatori și caracteristici

Funcționalități accesate:

- autentificare;
- creare cont;
- rezolvare quiz;
- vizualizare statistici;
- accesarea secțiunii Help.

Sistemul recunoaște o singură clasă de utilizatori: **Utilizatorul Simplu (Candidatul)**. Nu există un rol de administrator. Orice persoană care își creează un cont poate rezolva chestionare și își poate vedea propriul istoric. Toți utilizatorii au drepturi egale de acces la setul comun de întrebări

4. Mediu de operare

Programul a fost dezvoltat în limbajul **C#**, utilizând **.NET 8** și **Windows Forms** pentru interfața grafică. Aceasta este destinată să funcționeze într-un mediu desktop tipic, compatibil cu sistemele Windows moderne.

Specificații ale mediului de operare:

- **Platformă hardware:**
PC-uri, laptopuri sau terminale dotate cu arhitectură x64. Tablete sau dispozitive touchscreen cu Windows sunt de asemenea suportate.

- **Sistem de operare:**
 - Windows 10
 - Windows 11
- (Necesită .NET Runtime 8 instalat)

- **Framework necesar:**
 - .NET 8 SDK și/sau Runtime
 - Windows Forms support

5. Restricții privind proiectarea și implementarea

Aplicația este dezvoltată exclusiv pentru platforma Windows. Interfața grafică utilizează Windows Forms, ceea ce limitează compatibilitatea cu alte sisteme de operare.

Datele sunt stocate local în fișiere XML și TXT, fără utilizarea unui server de baze de date.

Pentru selectarea întrebărilor este utilizat șablonul de proiectare Strategy.

Codul respectă convențiile standard de dezvoltare C# și principiile programării orientate pe obiect.

Chestionarul este fixat pe un format standard: 26 de întrebări, doar categoria B, fără mod separat de învățare.

6. Documentația utilizatorului

Interfața dispune de un buton de „help” care oferă asistență rapidă și instrucțiuni utile pentru utilizare. Documentația a fost realizată cu ajutorul instrumentului HelpNDoc, ceea ce asigură un format organizat și ușor de parcurs.

7. Ipoteze și dependențe

În dezvoltarea și implementarea aplicației s-au făcut următoarele presupuneri și s-au identificat anumite dependențe care pot influența cerințele și funcționarea software-ului:

- Aplicația este dezvoltată pe platforma .NET (cu Windows Forms), care asigură compatibilitate și suport pe termen lung pentru mediul de execuție, facilitând actualizările și mentenanța ulterioară.
- Pentru generarea documentației utilizatorului și a funcției de help integrat, s-a utilizat aplicația terță HelpNDoc, ceea ce implică o dependență de această unealtă pentru actualizarea și întreținerea documentației.
- Hardware-ul și mediul de operare pe care va rula aplicația trebuie să respecte cerințele minime specifice platformei .NET, pentru a asigura funcționarea corectă și performanța optimă.
- Biblioteca și componentele externe folosite sunt compatibile cu versiunea curentă a .NET și cu mediul Windows Forms; eventuale modificări sau incompatibilități pot afecta funcționalitatea aplicației.

Se presupune că fișierul XML care conține baza de date de întrebări (`questions.xml`) și imaginile asociate întrebărilor sunt prezente în directorul de execuție al aplicației și nu sunt corupte. Alterarea manuală a structurii fișierului `statistics.txt` poate duce la erori de parsare în `StatisticsService`.

Orice schimbare în aceste condiții sau componente externe poate necesita modificări în cerințele sau implementarea aplicației.

C. Cerințe de interfață externă

1. Interfața cu utilizatorul

Aplicația **IaPermis** include mai multe interfețe grafice distincte, dezvoltate utilizând platforma **Windows Form**. Interfețele respectă o structură coerentă, intuitivă și prietenoasă, asigurând o experiență de utilizare eficientă.

Fereastra de autentificare:

- câmp username;
- câmp parolă;
- buton autentificare;
- buton creare cont;
- buton Help;

Fereastra principală

- buton „Începe Quiz”;
- buton „Vezi Statistici”;
- buton „Help”;
- buton „Delogare”.

Fereastra quiz

- întrebarea curentă;
- variantele de răspuns;
- imagine asociată întrebării;
- timpul rămas;
- numărul întrebării;
- numărul de greșeli;
- buton „Continuă”;
- buton Help
- buton „Părăsește progresul”
-

2. Interfața hardware

Aplicația **IaPermis** este proiectată să funcționeze pe dispozitive standard, compatibile cu Windows, inclusiv PC-uri și tablete.

Dispozitive suportate:

- PC / Laptop / Tabletă cu Windows 10 sau superior;
- Monitor, tastatură, mouse sau ecran tactil (în cazul tabletelor);

Aplicația nu necesită alte echipamente speciale și este optimizată pentru utilizarea atât pe stații fixe, cât și pe tablete în scopul mobilității sporite.

3. Interfața software

Aplicația este dezvoltată în mediul Microsoft Visual Studio, folosind limbajul C# și .NET 8.0 Windows Desktop Runtime, rulând pe sistemul de operare Windows 10/11. Pentru interfața grafică WinForms utilizează librăriile standard System.Data, System.Drawing și System.Windows.Forms, precum și alte librării C# uzuale (ex: System.Text, System.Linq, System.Collections.Generic).

Comunicarea între componentele aplicației se realizează prin apeluri de metode și evenimente, conform pattern-ului Strategy. În versiunea actuală, aplicația nu include comunicații în rețea sau cu alte servicii externe.

4. Interfața de comunicații

Aplicația IaPermis nu implică, în versiunea curentă, funcții de comunicare prin rețea, e-mail sau alte protocoale externe. Toate operațiunile se realizează local, fără schimb de mesaje cu servere sau alte dispozitive.

D. Funcțiile sistemului

1. Autentificare și înregistrare

1.1 Descriere și prioritate

Funcționalitatea permite utilizatorului să creeze un cont nou și să se autentifice în aplicație.

Prioritate: Ridică.

1.2 Secvențe stimul-răspuns

- Utilizatorul introduce username și parolă.
- Sistemul verifică datele.
- Dacă datele sunt valide, utilizatorul este autentificat.
- Dacă datele sunt invalide, sistemul afișează mesaj de eroare.

1.3 Cerințe funcționale

REQ-1: Sistemul trebuie să permită crearea unui cont nou.

REQ-2: Sistemul trebuie să valideze username-ul și parola.

REQ-3: Sistemul trebuie să permită autentificarea utilizatorilor existenți.

REQ-4: Sistemul trebuie să afișeze mesaje de eroare pentru autentificare invalidă.

2. Generarea unui quiz

2.1 Descriere și prioritate

Funcționalitatea permite generarea aleatorie a unui quiz format din 26 de întrebări.

Prioritate: Ridicăta.

2.2 Secvențe stimul-răspuns

- Utilizatorul apasă „Începe Quiz”.
- Sistemul citește întrebările din XML.
- Sistemul selectează aleatoriu 26 de întrebări.
- Quiz-ul este afișat utilizatorului.

2.3 Cerințe funcționale

REQ-1: Sistemul trebuie să citească întrebările din fișier XML.

REQ-2: Sistemul trebuie să selecteze întrebările utilizând Strategy Pattern.

REQ-3: Sistemul trebuie să afișeze întrebările în ordine.

REQ-4: Sistemul trebuie să afișeze imaginile asociate întrebărilor.

3. Rezolvarea chestionarului

3.1 Descriere și prioritate

Permite utilizatorului să răspundă la întrebări și să finalizeze quiz-ul.

Prioritate: Ridicăta.

3.2 Secvențe stimul-răspuns

- Sistemul afișează întrebarea.
- Utilizatorul selectează răspunsul.
- Timerul monitorizează timpul rămas.
- Dacă timpul expiră, sistemul trece la următoarea întrebare.
- La final se afișează scorul și calificativul obținut (admins/respins)

3.3 Cerințe funcționale

REQ-1: Sistemul trebuie să limiteze timpul unei întrebări la 30 secunde.

REQ-2: Sistemul trebuie să afișeze numărul de greșeli.

REQ-3: Sistemul trebuie să afișeze întrebarea curentă și progresul quiz-ului.

REQ-4: Sistemul trebuie să calculeze scorul final.

REQ-5: Sistemul trebuie să afișeze rezultatul final.

4. Vizualizarea statisticilor

4.1 Descriere și prioritate

Funcționalitatea permite salvarea și calcularea statisticilor utilizatorului.

Prioritate: Medie.

4.2 Secvențe stimul-răspuns

- Quiz-ul se finalizează.
- Sistemul salvează rezultatul.
- Utilizatorul accesează secțiunea „Statistici”.
- Sistemul afișează rata de promovare și scorul mediu.

4.3 Cerințe funcționale

REQ-1: Sistemul trebuie să salveze rezultatele utilizatorului.

REQ-2: Sistemul trebuie să calculeze rata de promovare.

REQ-3: Sistemul trebuie să calculeze scorul mediu.

5. Vizualizare rezultate quiz

5.1 Descriere și prioritate

Deși funcționalitatea de bază este rularea quiz-ului, monitorizarea progresului și vizualizarea statisticilor reprezintă o cerință esențială pentru utilizatori în procesul de învățare și evaluare a gradului de pregătire pentru examenul auto real.

Prioritate: Ridică

5.2 Secvențe stimul-răspuns

- Quiz-ul se finalizează
- Dacă se greșesc mai mult de 4 întrebări, pe ecran se va afișa cu roșu „Ai picat”
- Dacă utilizatorul ajunge până la sfârșitul testului, se va afișa cu verde „Ai trecut”

5.3 Cerințe funcționale

REQ-1: Sistemul trebuie să verifice automat după fiecare întrebare dacă numărul de greșeli a depășit limita maximă permisă (4 greșeli)

REQ-2: Sistemul trebuie să oprească imediat chestionarul în momentul în care utilizatorul este respins, fără a-l mai obliga să răspundă la întrebările rămase..

REQ-3: Dacă testul este promovat, interfața grafică trebuie să coloreze ecranul în **verde** și să afișeze clar textul de confirmare („Ai trecut”).

REQ-4: Dacă testul este picat, interfața grafică trebuie să coloreze ecranul în **roșu** și să afișeze textul de avertizare („Ai picat”).

E. Alte cerințe non-funcționale

1. Cerințe de performanță

Aplicația trebuie să răspundă la acțiunile utilizatorului în maximum 1 secundă.

Întrebările trebuie încărcate complet în maximum 2 secunde.

Trecerea la următoarea întrebare trebuie realizată instantaneu.

2. Cerințe de siguranță

Aplicația trebuie să trateze excepțiile generate de:

- lipsa fișierelor;
- fișiere corupte;
- date invalide.

Mesajele afișate utilizatorului trebuie să fie clare și fără detalii tehnice.

3. Cerințe de securitate

Datele utilizatorilor sunt stocate local. Aplicația nu transmite date către servicii externe. Parolele trebuie gestionate într-un mod sigur.

4. Atribute de calitate a software-ului

Aplicația oferă o interfață intuitivă, optimizată pentru utilizatori fără pregătire tehnică. Este stabilă și fiabilă, fără blocaje în utilizare normală. Codul este modular, bine organizat și ușor de întreținut. Funcționează pe orice sistem Windows cu .NET 8, fără alte dependențe. Fiecare funcționalitate poate fi testată independent, iar aplicația gestionează elegant erorile de la nivelul UI sau al bazei de date.

5. Reguli de business

Un quiz conține exact 26 de întrebări. Fiecare întrebare are un timp limită de 30 secunde.

Întrebările sunt selectate aleatoriu.

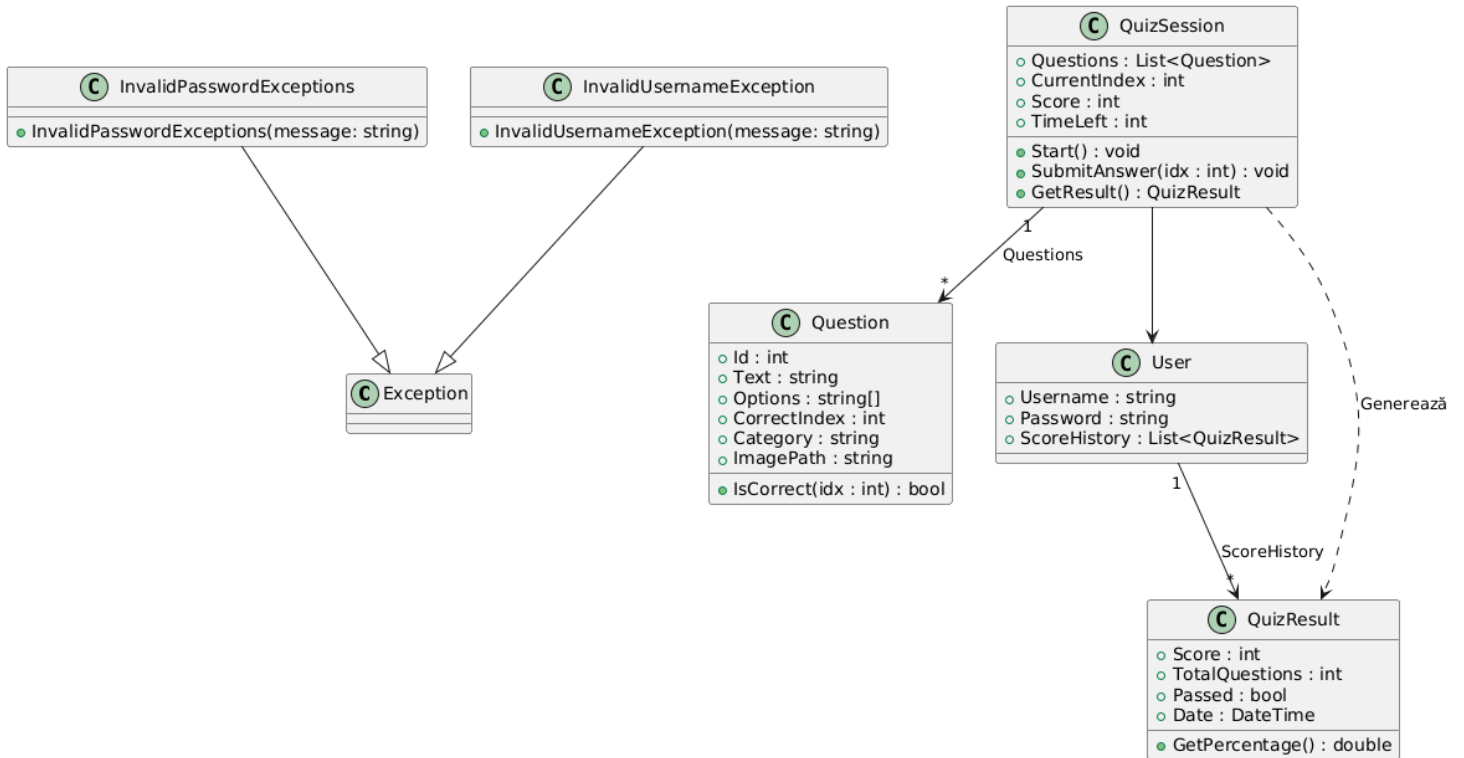
Statisticile sunt asociate utilizatorului autentificat.

Utilizatorul trebuie să fie autentificat pentru a accesa quiz-ul și statisticile

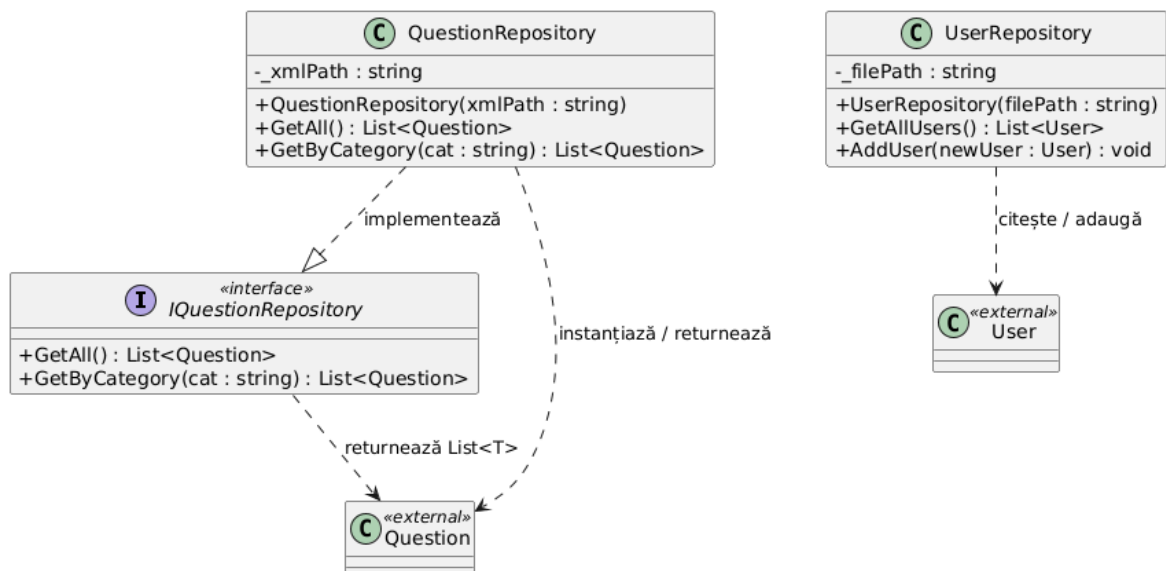
II. Diagrame UML

A. Diagrama de clase

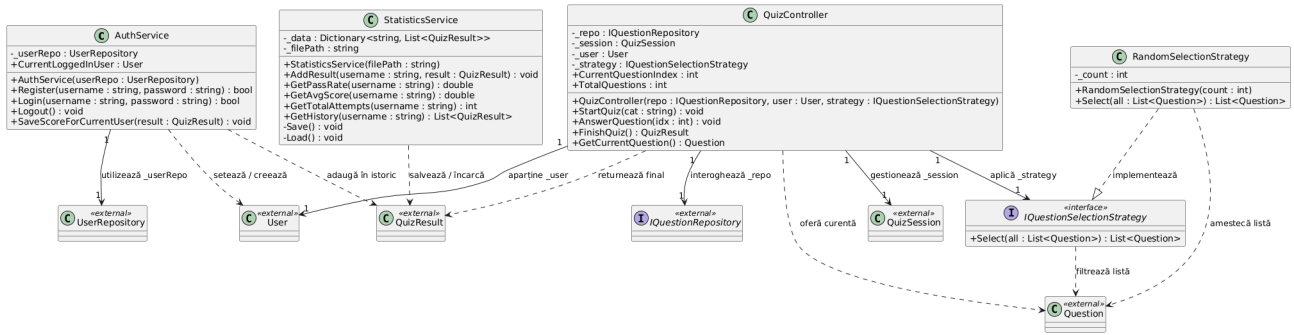
1. QuizAppCore



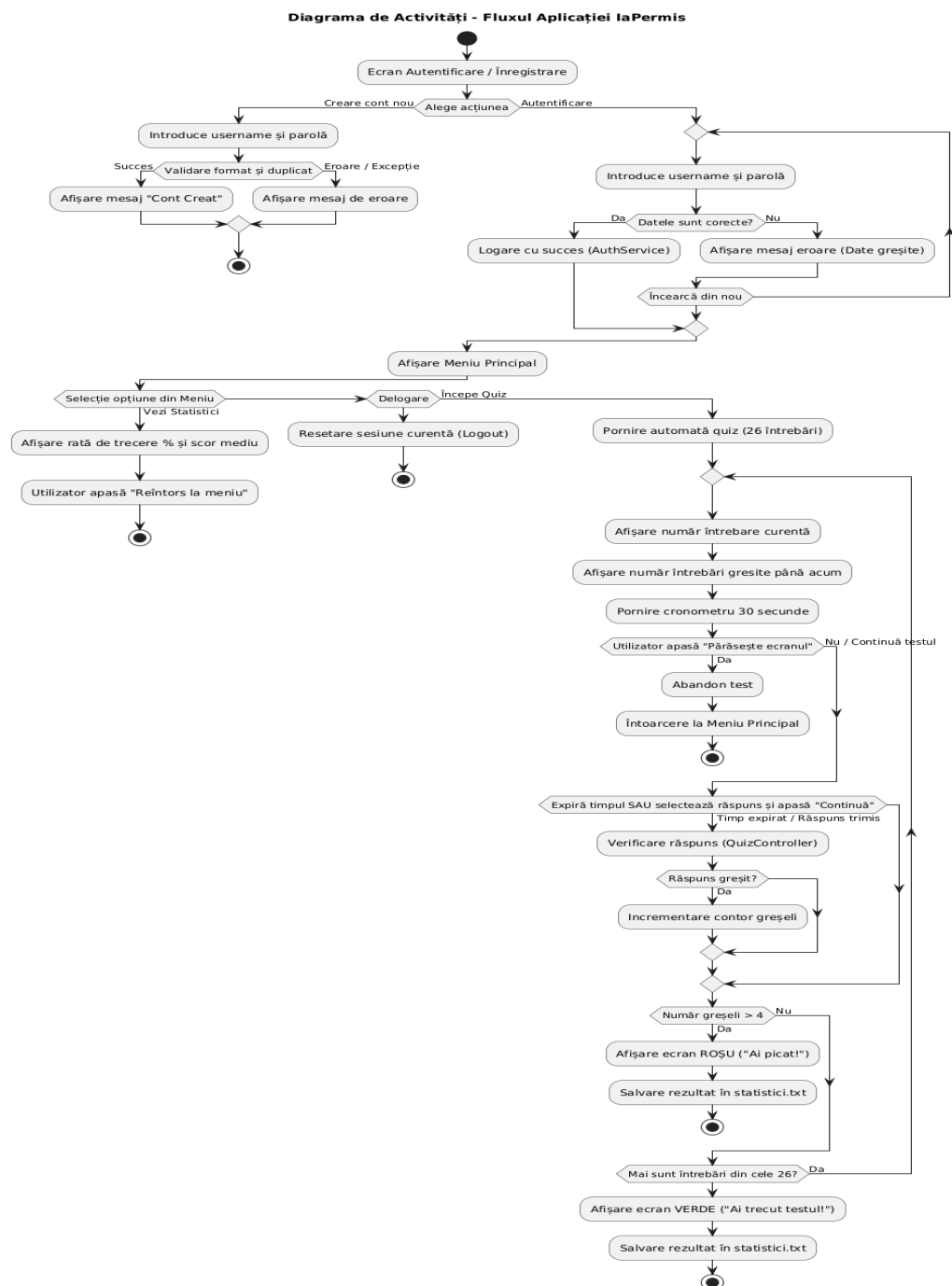
2. QuizAppData



3. QuizAppServices

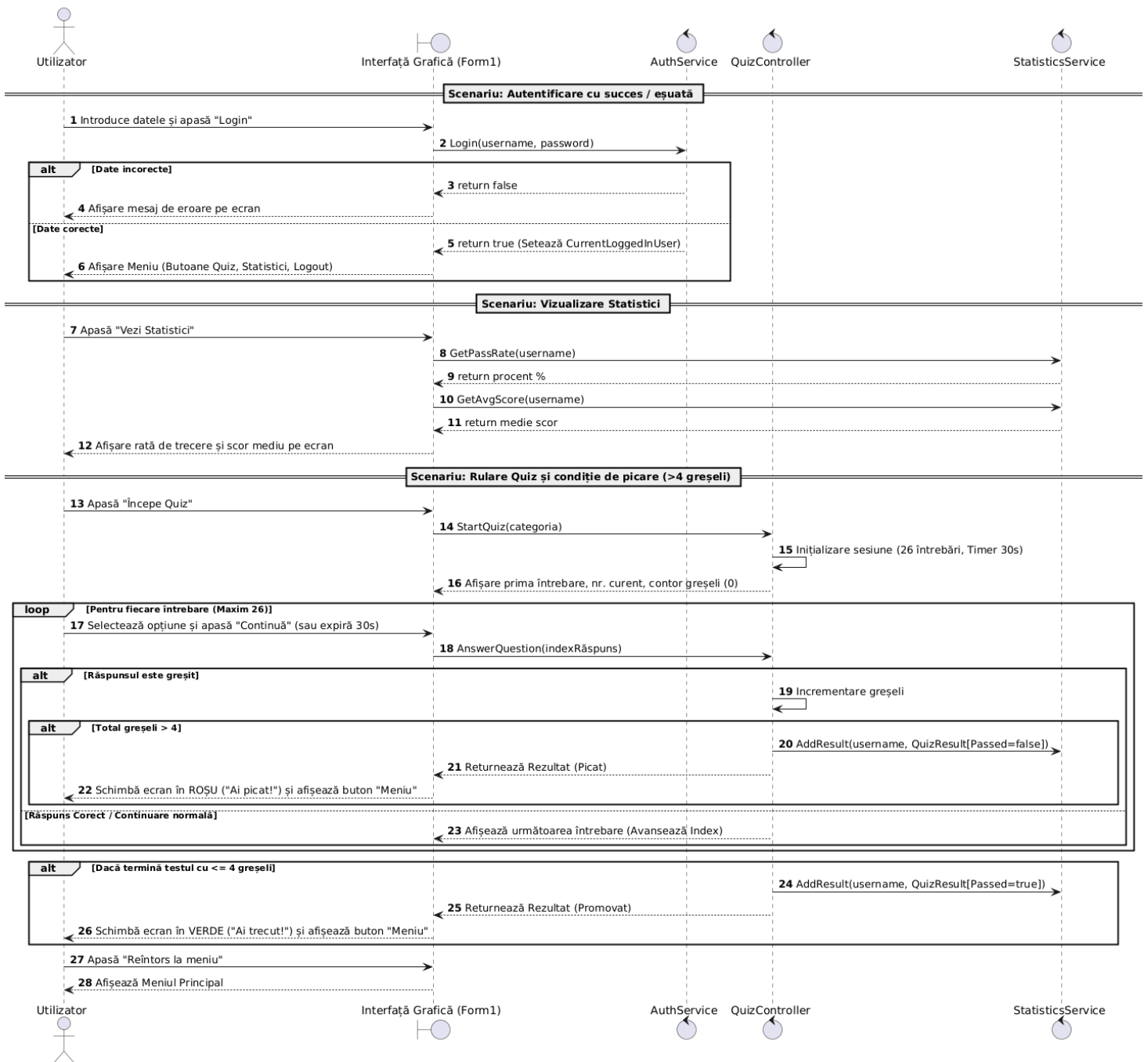


B. Diagrama de activități

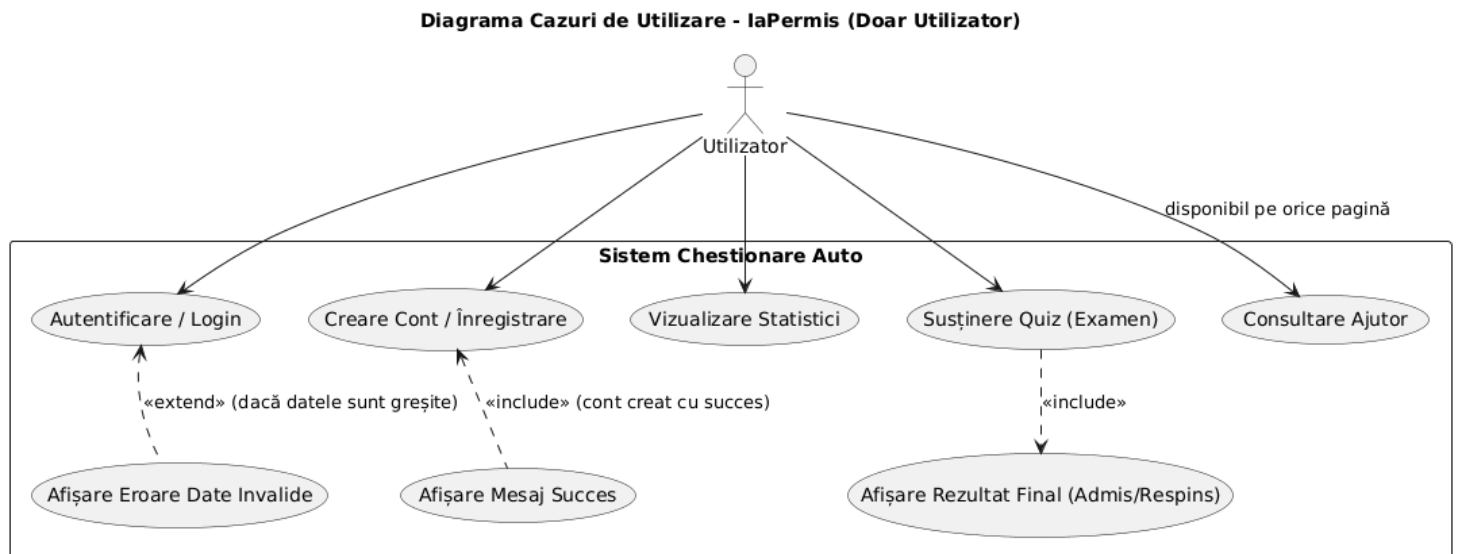


C. Diagrama de secvențe

Diagrama de Secvență - Autentificare și Desfășurare Quiz

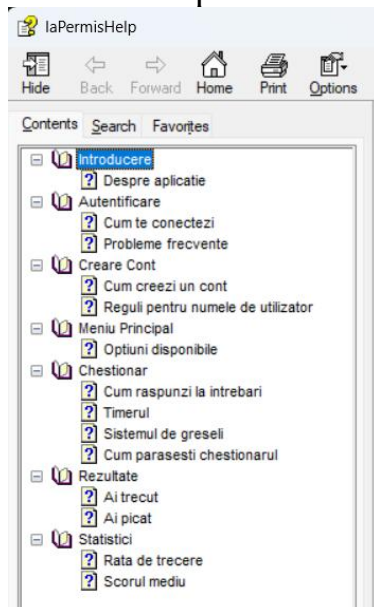


D. Diagrama de cazuri de utilizare



III. Modul de utilizare a programului

Structura Help:



1. Autentificare și creare cont

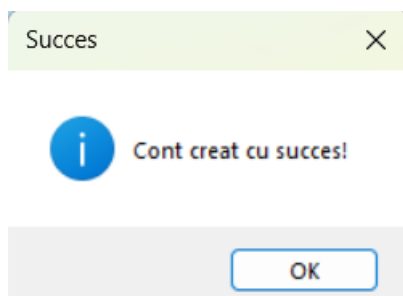
La pornirea aplicației utilizatorul poate:

- introduce username și parolă;

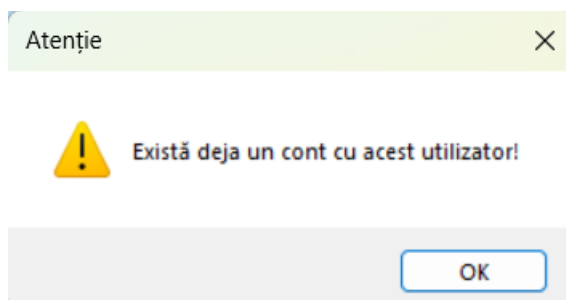
- crea un cont nou;
- se poate autentifica la un cont existent
- accesa secțiunea Help.



La înregistrare, se va afișa mesajul „Cont creat cu succes”



Dacă vrem să creăm un nou cont cu un nume de utilizator existent, se va afișa un mesaj de eroare



2. Meniu principal

După autentificare utilizatorul poate apăsa butonul „Începe Quiz” sau „Vezi statistici”, de asemenea se poate deloga sau poate vizualiza help.



3. Chestionar

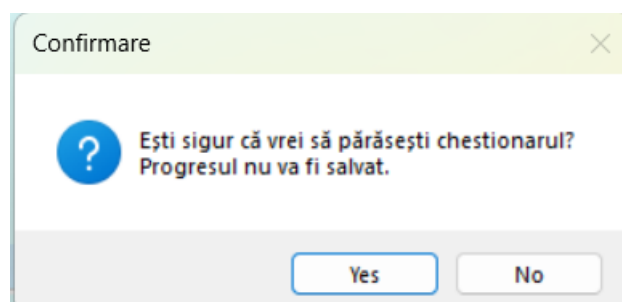
Sistemul generează automat un chestionar de 26 de întrebări. În partea de sus a ferestrei va exista un progressBar care se va umple complet navigând prin toate întrebările. Există de asemenea și un buton „părăsește progresul” pentru a opri testul.

Pentru fiecare întrebare:

- se afișează timpul rămas;
- se afișează numărul întrebării;
- se afișează numărul de greșeli;
- utilizatorul selectează răspunsul.



Există și un buton „părăsește programul” care va încheia quiz-ul, iar pe ecran va apărea un mesaj



4. Vizualizarea statisticilor

Utilizatorul poate vedea:

- rata de trecere;
- scorul mediu.



5. Vizualizare rezultat quiz

Dacă utilizatorul greșește mai mult de 4 întrebări, testul se va termina automat și pe ecran va fi afișat mesajul „Ai picat, mai învață..”, iar dacă utilizatorul trece testul, se va afișa „Felicitări, ai trecut!” și scorul obținut.



IV. Anexă – Cod sursă relevant

1. Strategy Pattern

```

/*****
* Fișier:      IQuestionSelectionStrategy.cs
* Autor:      David Bianca
* Data:      Mai 2026
* Proiect:    IaPermis - Chestionare Auto
* Funcționalitate: Definește strategia de selecție a întrebărilor
*              pentru quiz.
*****/

using QuizAppCore;
using System.Collections.Generic;

namespace QuizAppServices
{
    /// <summary>
    /// Interfață care definește contractul pentru strategiile de selecție a întrebărilor.
    /// Permite schimbarea modului de selectare fără a modifica restul aplicației.
    /// </summary>
    public interface IQuestionSelectionStrategy
    {
        /// <summary>
        /// Selectează un subset de întrebări din lista completă disponibilă.
        /// </summary>
        /// <param name="all">Lista tuturor întrebărilor disponibile.</param>
        /// <returns>Lista de întrebări selectate pentru quiz.</returns>
        List<Question> Select(List<Question> all);
    }
}

/*****
* Fișier:      RandomSelectionStrategy.cs
* Autor:      David Bianca
* Data:      Mai 2026
* Proiect:    IaPermis - Chestionare Auto
* Funcționalitate: Selectează aleatoriu întrebările utilizate în quiz.
*****/

using QuizAppCore;
using System;
using System.Collections.Generic;
using System.Linq;

namespace QuizAppServices
{
    public class RandomSelectionStrategy : IQuestionSelectionStrategy
    {
        // Numarul de intrebari de selectat pentru un quiz
        private int _count;

        /// <summary>
        /// Initializeaza strategia cu numarul dorit de intrebari.
        /// Implicit se selecteaza 26 de intrebari (standard examen auto).
        /// </summary>
        /// <param name="count">Numarul de intrebari de selectat.</param>
        public RandomSelectionStrategy(int count = 26)
        {
            _count = count;
        }
    }
}

```

```

    }

    /// <summary>
    /// Selectează aleatoriu un număr de întrebări din lista furnizată.
    /// Dacă lista are mai puține întrebări decât numărul cerut, returnează toate.
    /// </summary>
    /// <param name="all">Lista completă de întrebări disponibile.</param>
    /// <returns>Lista de întrebări selectate aleatoriu.</returns>
    public List<Question> Select(List<Question> all)
    {
        if (all == null || all.Count == 0)
            throw new ArgumentException("Lista de întrebări este goală.");

        // Amestecăm lista folosind un număr random și luăm primele _count elemente
        var rng = new Random();
        return all.OrderBy(q => rng.Next()).Take(_count).ToList();
    }
}

}

/*****
* Fișier:      QuizController.cs
* Autor:       David Bianca
* Data:        Mai 2026
* Proiect:     IaPermis - Chestionare Auto
* Funcționalitate: Controlează desfășurarea quiz-ului, gestionarea
*                întrebărilor și calcularea rezultatului final.
*****/

using QuizAppCore;
using QuizAppData;
using System;
using System.Collections.Generic;
using System.Linq;

namespace QuizAppServices
{
    public class QuizController
    {
        // Repository-ul din care se încarcă întrebările
        private IQuestionRepository _repo;

        // Sesiunea activă a quiz-ului curent
        private QuizSession _session;

        // Utilizatorul care susține quiz-ul
        private User _user;

        // Strategia folosită pentru selectarea întrebărilor
        private IQuestionSelectionStrategy _strategy;

        /// <summary>
        /// Indexul întrebării curente în cadrul sesiunii active.
        /// </summary>
        public int CurrentQuestionIndex => _session.CurrentIndex;

        /// <summary>
        /// Numărul total de întrebări din sesiunea curentă.
        /// </summary>
        public int TotalQuestions => _session.Questions.Count;

        /// <summary>
        /// Initializează controller-ul de quiz cu dependentele necesare.
        /// Dacă nu este furnizată o strategie, se folosește selecția aleatorie implicită.

```

```

    /// </summary>
    /// <param name="repo">Repository-ul de intrebari.</param>
    /// <param name="user">Utilizatorul care sustine quiz-ul.</param>
    /// <param name="strategy">Strategia de selectie a intrebarilor
(optional).</param>
    public QuizController(IQuestionRepository repo, User user,
        IQuestionSelectionStrategy strategy = null)
    {
        _repo = repo;
        _user = user;
        _session = new QuizSession();
        // Daca nu se specifica o strategie, se foloseste cea aleatorie cu 26 de
intrebari
        _strategy = strategy ?? new RandomSelectionStrategy();
    }

```

2. Autentificare & inregistrare

```

    /// <summary>
    /// Valideaza campurile de login si autentifica utilizatorul.
    /// In caz de succes, navigheaza catre panoul de meniu principal.
    /// </summary>
    private async void buttonLoginAutentificare_Click(object sender, EventArgs e)
    {
        string user = textBoxLoginUser.Text.Trim();
        string pass = textBoxLoginPassword.Text;

        // Verificam ca ambele campuri sunt completate
        if (string.IsNullOrEmpty(user) || string.IsNullOrEmpty(pass))
        {
            MessageBox.Show("Completează utilizatorul și parola!", "Atenție",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        try
        {
            bool ok = _authService.Login(user, pass);
            if (!ok)
            {
                MessageBox.Show("Utilizator sau parolă incorecte!", "Eroare",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
                return;
            }

            // Login reusit – setam utilizatorul curent si navigam la meniu
            _currentUser = user;
            labelHiUser.Text = $"Salut, {_currentUser}!";
            await ShowPanel(panelMenu);
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Eroare la autentificare:\n{ex.Message}", "Eroare",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }

    /// <summary>
    /// Navigheaza catre panoul de inregistrare cont nou.
    /// </summary>
    private async void buttonLoginCreateAccount_Click(object sender, EventArgs e)
    {

```

```

    await ShowPanel(panelRegister);
}

// HANDLERS PANEL REGISTER

/// <summary>
/// Valideaza datele introduse, creeaza contul nou si logheaza automat utilizatorul.
/// Prinde exceptiile specifice pentru username si parola invalide.
/// </summary>
private async void buttonRegister_Click(object sender, EventArgs e)
{
    string user = textBoxRegisterUser.Text.Trim();
    string pass = textBoxRegisterPassword.Text;

    if (string.IsNullOrEmpty(user) || string.IsNullOrEmpty(pass))
    {
        MessageBox.Show("Completează toate câmpurile!", "Atenție",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    try
    {
        bool ok = _authService.Register(user, pass);
        if (!ok)
        {
            MessageBox.Show("Există deja un cont cu acest utilizator!", "Atenție",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        // După înregistrare reușită, logăm automat
        _authService.Login(user, pass);
        _currentUser = user;

        MessageBox.Show("Cont creat cu succes!", "Succes",
            MessageBoxButtons.OK, MessageBoxIcon.Information);

        labelHiUser.Text = $"Salut, {_currentUser}!";
        await ShowPanel(panelMenu);
    }
    catch (InvalidPasswordExceptions ex)
    {
        MessageBox.Show(ex.Message, "Parolă invalidă",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    catch (InvalidUsernameException ex)
    {
        // Excepția specifică definită (caractere speciale, username scurt)
        MessageBox.Show(ex.Message, "Utilizator invalid",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Eroare la crearea contului:\n{ex.Message}", "Eroare",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

3. Cod testare

```
[TestMethod]
public void SubmitAnswer_CorrectAnswer_IncrementsScore()
{
    var session = new QuizSession();
    session.Questions.Add(new Question
    {
        Text = "Test?",
        Options = new[] { "A", "B", "C", "D" },
        CorrectIndex = 0
    });
    session.Start();
    session.SubmitAnswer(0);
    Assert.AreEqual(1, session.Score);
}
```

```
[TestMethod]
public void SubmitAnswer_WrongAnswer_ScoreUnchanged()
{
    var session = new QuizSession();
    session.Questions.Add(new Question
    {
        Text = "Test?",
        Options = new[] { "A", "B", "C", "D" },
        CorrectIndex = 0
    });
    session.Start();
    session.SubmitAnswer(2);
    Assert.AreEqual(0, session.Score);
}
```

```
[TestMethod]
public void GetResult_Score22_PassedIsTrue()
{
    var session = new QuizSession();
    for (int i = 0; i < 22; i++)
        session.Questions.Add(new Question
        {
            Options = new[] { "A", "B", "C", "D" },
            CorrectIndex = 0
        });
    session.Start();
    for (int i = 0; i < 22; i++)
        session.SubmitAnswer(0);
    var result = session.GetResult();
    Assert.IsTrue(result.Passed);
}
```

```
[TestMethod]
public void GetResult_Score21_PassedIsFalse()
{
    var session = new QuizSession();
    for (int i = 0; i < 21; i++)
        session.Questions.Add(new Question
        {
            Options = new[] { "A", "B", "C", "D" },
            CorrectIndex = 0
        });
    session.Start();
    for (int i = 0; i < 21; i++)
        session.SubmitAnswer(0);
    var result = session.GetResult();
}
```

```

        Assert.IsFalse(result.Passed);
    }

    [TestMethod]
    public void GetPercentage_FullScore_Returns100()
    {
        // Arrange
        var result = new QuizResult { Score = 26, TotalQuestions = 26 };
        // Act & Assert
        Assert.AreEqual(100.0, result.GetPercentage());
    }
}

public class AuthServiceTests
{

    [TestMethod]
    public void Login_ValidCredentials_ReturnsTrue()
    {
        var repo = new UserRepository("test_users2.txt");
        var auth = new AuthService(repo);
        auth.Register("userLogin", "pass123");
        bool result = auth.Login("userLogin", "pass123");
        Assert.IsTrue(result);
    }

    [TestMethod]
    public void Login_WrongPassword_ReturnsFalse()
    {
        var repo = new UserRepository("test_users3.txt");
        var auth = new AuthService(repo);
        auth.Register("userWrong", "corecta");
        bool result = auth.Login("userWrong", "gresita");
        Assert.IsFalse(result);
    }

    [TestMethod]
    [ExpectedException(typeof(InvalidUsernameException))]
    public void Register_ShortUsername_ThrowsException()
    {
        var repo = new UserRepository("test_users4.txt");
        var auth = new AuthService(repo);
        auth.Register("ab", "parola123");
    }

    [TestMethod]
    [ExpectedException(typeof(InvalidUsernameException))]
    public void Register_UsernameWithSpecialChars_ThrowsException()
    {
        var repo = new UserRepository("test_users5.txt");
        var auth = new AuthService(repo);
        auth.Register("user@bad", "parola123");
    }

    [TestMethod]
    public void Logout_SetsCurrentUserNull()
    {
        var repo = new UserRepository("test_users6.txt");
        var auth = new AuthService(repo);
        auth.Register("userLogout", "pass");
        auth.Login("userLogout", "pass");
        auth.Logout();
    }
}

```

```

        Assert.IsNull(auth.CurrentLoggedInUser);
    }

    [TestMethod]
    public void Register_DuplicateUsername_ReturnsFalse()
    {
        // Arrange
        var repo = new UserRepository("test_users7.txt");
        var auth = new AuthService(repo);
        auth.Register("userDuplikat", "pass123");
        // Act - al doilea Register cu acelasi username
        bool result = auth.Register("userDuplikat", "altaparola");
        // Assert
        Assert.IsFalse(result);
    }

    [TestMethod]
    public void Login_NonExistentUser_ReturnsFalse()
    {
        // Arrange
        var repo = new UserRepository("test_users8.txt");
        var auth = new AuthService(repo);
        // Act - user care nu exista
        bool result = auth.Login("userInexistent999", "oriceparola");
        // Assert
        Assert.IsFalse(result);
    }

```

4. Timer

```

/// <summary>
/// Configureaza timer-ul de intrebare: interval de 1 secunda
/// si asociaza handler-ul de tick.
/// </summary>
private void SetupTimer()
{
    _questionTimer = new System.Windows.Forms.Timer();
    _questionTimer.Interval = 1000;
    _questionTimer.Tick += QuestionTimer_Tick;
}

/// <summary>
/// Handler apelat la fiecare secunda de catre timer.
/// Decrementeaza cronometrul si trateaza expirarea timpului.
/// </summary>
private void QuestionTimer_Tick(object sender, EventArgs e)
{
    _secondsLeft--;
    UpdateTimerDisplay();

    if (_secondsLeft <= 0)
    {
        _questionTimer.Stop();
        OnTimeExpired();
    }
}

/// <summary>
/// Actualizeaza label-ul cu timpul ramas si ii schimba culoarea
/// in functie de urgenta: verde > 15s, portocaliu > 7s, rosu <= 7s.
/// </summary>
private void UpdateTimerDisplay()
{
    labelQuizTimeValue.Text = $"{_secondsLeft}s";
}

```

```

if ( _secondsLeft > 15)
    labelQuizTimeValue.ForeColor = Color.DarkGreen;
else if ( _secondsLeft > 7)
    labelQuizTimeValue.ForeColor = Color.DarkOrange;
else
    labelQuizTimeValue.ForeColor = Color.Crimson;
}

/// <summary>
/// Tratarea expirarii timpului pentru o intrebare.
/// Trimite un index invalid (-1) ca raspuns gresit si verifica limita de greseli.
/// </summary>
private void OnTimeExpired()
{
    // Index -1 → IsCorrect returneaza false → raspuns gresit automat
    if ( _quizController != null)
        _quizController.AnswerQuestion(-1); // index invalid → IsCorrect returnează false

    _wrongAnswers++;
    labelQuizWrongAnswers.Text = $"Greșeli: {_wrongAnswers}/3";

    // Daca s-au acumulat 3 greseli, quiz-ul se termina imediat
    if ( _wrongAnswers >= 3)
    {
        EndQuiz();
        return;
    }

    NextQuestion();
}

/// <summary>
/// Reseteaza si porneste timer-ul pentru o intrebare noua.
/// </summary>
private void StartQuestionTimer()
{
    _secondsLeft = 30;
    UpdateTimerDisplay();
    _questionTimer.Start();
}

```

5. Logica de quiz

```

/// <summary>
/// Incarca si afiseaza intrebarea curenta din sesiune:
/// text, variante de raspuns, imagine si progress bar.
/// Reseteaza starea de raspuns si porneste timer-ul.
/// </summary>
private async void LoadQuestion()
{
    _answerSelected = false;
    _processingAnswer = false;
    buttonQuizNext.Enabled = false;

    Question q = null;
    try
    {
        q = _quizController?.GetCurrentQuestion();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Eroare la încărcarea întrebării:\n{ex.Message}",
            "Eroare", MessageBoxButtons.OK, MessageBoxIcon.Error);
        await ShowPanel(panelMenu);
    }
}

```

```

        return;
    }

    // Daca nu mai sunt intrebari, finalizam quiz-ul
    if (q == null)
    {
        EndQuiz();
        return;
    }

    int currentDisplayIndex = _quizController.CurrentQuestionIndex;
    int total = _quizController.TotalQuestions;

    progressBarQuiz.Maximum = total;
    progressBarQuiz.Value = currentDisplayIndex;

    labelQuizQuestionNumber.Text = $"Număr: {currentDisplayIndex + 1}/{total}";
    labelQuizWrongAnswers.Text = $"Greșeli: {_wrongAnswers}/3";

    // Afisam textul intrebarii
    labelQuizQuestion.Text = q.Text;
    labelQuizQuestion.Visible = true;
    labelQuizQuestion.BringToFront();

    // Populam butoanele cu variantele de raspuns
    SetAnswerButtons(q.Options);

    // Incarcam imaginea asociata (daca exista)
    LoadQuestionImage(q.ImagePath);

    StartQuestionTimer();
    labelQuizQuestion.Visible = true;
    labelQuizQuestion.BringToFront();
    panelQuiz.Refresh();
}

/// <summary>
/// Seteaza textul si vizibilitatea butoanelor de raspuns
/// in functie de variantele disponibile pentru intrebarea curenta.
/// Ajusteaza si dimensiunea fontului in functie de lungimea textului.
/// </summary>
/// <param name="answers">Array-ul cu variantele de raspuns.</param>
private void SetAnswerButtons(string[] answers)
{
    Button[] buttons = { buttonQuizAnswer1, buttonQuizAnswer2,
                        buttonQuizAnswer3, buttonQuizAnswer4 };

    for (int i = 0; i < buttons.Length; i++)
    {
        if (i < answers.Length && !string.IsNullOrEmpty(answers[i]))
        {
            // Varianta exista - o afisam si o activam
            buttons[i].Text = answers[i];
            buttons[i].Enabled = true;
            buttons[i].Visible = true;
            buttons[i].BackColor = Color.LightCyan;
            AdjustButtonFont(buttons[i]);
        }
        else
        {
            // Varianta lipseste - ascundem butonul
            buttons[i].Text = "";
            buttons[i].Enabled = false;
        }
    }
}

```

```

        buttons[i].Visible = false;
    }
}

}

/// <summary>
/// Finalizeaza sesiunea de quiz: opreste timer-ul, calculeaza rezultatul,
/// il salveaza in istoricul utilizatorului si afiseaza panoul de rezultate.
/// </summary>
private async void EndQuiz()
{
    _questionTimer.Stop();

    QuizResult result = null;
    try
    {
        result = _quizController?.FinishQuiz();

        // Salvăm rezultatul în istoricul utilizatorului curent
        if (result != null)
            _authService?.SaveScoreForCurrentUser(result);

        // Actualizăm statisticile
        if (result != null && _authService?.CurrentLoggedInUser != null)
            _statsService?.AddResult(_authService.CurrentLoggedInUser.Username, result);
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Eroare la finalizarea quiz-ului:\n{ex.Message}",
            "Eroare", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    int score = result?.Score ?? 0;
    ShowResults(score, result?.TotalQuestions ?? _totalQuestions);
    await ShowPanel(panelResults);
}

/// <summary>
/// Afiseaza rezultatul final in panoul de rezultate.
/// Coloreaza fundalul in verde (promovat) sau rosu (picat)
/// in functie de pragul de 22 de raspunsuri corecte.
/// </summary>
/// <param name="score">Numarul de raspunsuri corecte obtinute.</param>
/// <param name="total">Numarul total de intrebari din sesiune.</param>
private void ShowResults(int score, int total)
{
    labelResultsNumber.Text = $"Rezultat: {score}/{total}";
    labelResultsHiUser.Text = $"Salut, {_currentUser}!";

    bool passed = score >= 22;

    if (passed)
    {
        groupBoxResults.BackColor = Color.YellowGreen;
        labelResultsMessage.Text = "Felicitări, ai trecut!";
        labelResultsMessage.ForeColor = Color.DarkGreen;
    }
    else
    {
        groupBoxResults.BackColor = Color.Crimson;
        labelResultsMessage.Text = "Ai picat, mai învață..";
        labelResultsMessage.ForeColor = Color.White;
    }
}
}

```

Rezultatul testelor:

The image displays three screenshots of the Visual Studio Test Explorer interface, showing the results of a test run for a project named 'QuizAppTests'. The test run finished with 39 tests passed, 0 failed, and 0 skipped, taking 2.3 seconds.

Test Explorer Screenshot 1 (Top): Shows a list of tests grouped under 'QuizResultTests (12)', 'QuizSessionTests (6)', and 'StatisticsServiceTests (3)'. The 'Group Summary' on the right indicates 'Tests in group: 39' and 'Total Duration: 325 ms', with 'Outcomes' showing '39 Passed'.

Test Explorer Screenshot 2 (Middle): Shows a different view of the test results, with 'QuizResultTests (12)' expanded. The 'Group Summary' on the right remains the same, indicating '39 Passed'.

Test Explorer Screenshot 3 (Bottom): Shows a third view of the test results, with 'QuizAppTests (39)' expanded. The 'Group Summary' on the right remains the same, indicating '39 Passed'.

V. Listă task-uri îndeplinite

1. Salaru Ioana: Interfața Grafică (GUI) și Sistemul de Asistență (Help)

Responsabilități principale:

- **Dezvoltarea Interfeței Grafice Utilizator (GUI):** Proiectarea și implementarea completă a componentelor vizuale utilizând tehnologia *C# Windows Forms* (`Form1.cs`).
- **Integrarea Resurselor Vizuale:** Gestionarea elementelor grafice din aplicație, inclusiv imaginile de fundal (`background.png`) și încărcarea dinamică a imaginilor specifice întrebărilor de legislație rutieră.
- **Sistemul de Help (Asistență locală):** Conceperea, redactarea și integrarea fișierului de documentație compilat `IaPermisHelp.chm`. Conținutul este structurat pe înțelesul unui utilizator final non-tehnic, explicând pașii de utilizare a aplicației, semnificația elementelor de pe ecran și modul de interpretare a rezultatelor.

2. David Bianca - Vasilica: Arhitectură, Modularizare (DLL) și Șabloane de Proiectare

Responsabilități principale:

- **Modularizarea soluției prin DLL-uri:** Separarea riguroasă a claselor și a logicii de business în module independente. Pentru a asigura decuplarea codului, soluția a fost împărțită în biblioteci de clase (Class Libraries) compilate ca fișiere `.dll` separate:
 - `QuizAppCore.dll` – Conține entitățile de bază ale domeniului.
 - `QuizAppData.dll` – Gestionează accesul la fișiere (stocarea XML și TXT).
 - `QuizAppServices.dll` – Încorporează logica algoritmilor și serviciile aplicației.
- **Implementarea Șablonului de Proiectare (Design Pattern):** Integrarea design pattern-ului **Strategy** prin intermediul interfeței `IQuestionSelectionStrategy` și a clasei `RandomSelectionStrategy`. Acest șablon permite modificarea sau extinderea dinamică a modului în care sunt alese și livrate întrebările către utilizator, fără a altera structura existentă a codului.
- **Tratarea Defensivă a Excepțiilor:** Implementarea mecanismelor de gestionare a erorilor în straturile de date și servicii. Au fost create clase de excepții custom (`InvalidUsernameException`, `InvalidPasswordExceptions`) pentru a intercepta și trata elegant stările invalide ale sistemului (lipsa fișierelor de date, credențiale incorecte), prevenind blocarea aplicației.

3. Vrinceanu Iustina Maria: Testare Unitară, Documentarea Codului și Standarde de Scriere

Responsabilități principale:

- **Scrierea Cazurilor de Test (Unit Testing):** Dezvoltarea proiectului `QuizAppTests (UnitTest1.cs)` cuprinzând un pachet de **minimum 20 de cazuri de testare unitară**.
- **Asigurarea Standardelor de Cod:** Verificarea și refactorizarea întregului cod sursă din soluție pentru a respecta convențiile de numire (PascalCase, camelCase) și bunele practici de programare orientată obiect stabilite în laborator.
- **Documentarea Surselor și Antetele de Fișier:** Comentarea integrală a metodelor și claselor folosind tag-uri XML (`<summary>`, `<param>`, `<returns>`). Inserarea obligatorie a unui antet standardizat la începutul fiecărui fișier sursă, specificând: numele fișierului, autorul, data creării, proiectul de apartenență și funcționalitatea detaliată a codului respectiv.

4. Panaite Elena Alexandra: Documentația Tehnică și Diagramele UML

Responsabilități principale:

- **Redactarea Documentului SRS (Software Requirements Specification):** Crearea specificațiilor tehnice ale sistemului respectând standardul internațional *IEEE 830*, adaptat arhitecturii aplicației „IaPermis”.
- **Modelarea Vizuală (Diagrame UML):** Proiectarea și desenarea diagramelor fundamentale pentru înțelegerea arhitecturii software:
 - *Diagrama Cazurilor de Utilizare (Use Case Diagram):* Evidențiază acțiunile unice ale utilizatorului (autentificare, rezolvare quiz, vizualizare statistici).
 - *Diagrama de Clase (Class Diagram):* Ilustrează structura claselor din cele 5 proiecte și relațiile dintre ele.
 - *Diagrama de Activități (Activity Diagram):* Descrie fluxul logic al unui test, inclusiv condiția de timp (30s) și evaluarea finală.
 - *Diagrama de Secvență (Sequence Diagram):* Modelează interacțiunea în timp între GUI, Controller, Servicii și Repository.
- **Asamblarea și Formatarea Documentației Finale:** Tehnoredactarea raportului final de **minimum 10 pagini A4**, utilizând fontul *Times New Roman*, *mărimea 12*, *spațiere la 1 rând*. Integrarea capturilor de ecran din interfață și organizarea Anexei ce conține blocurile critice de cod sursă (formatate cu fontul *Consolas*, *mărimea 10*).